

System Architecture Document

Tango - YouTube to Anki Vocabulary Pipeline

Version 0.4.0 | July 2026

1. System Overview

Tango is a single-machine Python pipeline. It runs as a command-line tool, processes one YouTube video per invocation, and writes one .apkg file per run. All state is stored in a local SQLite database. The system has no server component in v0.4.0.

2. Component Architecture

The system is composed of nine modules in a linear pipeline. Each module has a single responsibility and communicates with adjacent modules via typed Python objects.

config.py

Loads all constants from environment variables and the .env file. All other modules import from config rather than reading environment variables directly. This centralises configuration and makes the system testable by monkeypatching config values.

language.py

Resolves the BCP-47 language code from the --language flag or the deck name. Contains the 40-language name mapping and the partial BCP-47 matching logic for transcript selection. Handles the preference for manually created transcripts over auto-generated ones.

transcript.py

Wraps youtube-transcript-api. Accepts a video ID, builds the proxy configuration from environment variables, lists available transcripts, selects the best match via language.py, fetches the snippet data, cleans the text, and returns a structured result containing the full text and snippet dictionary.

nlp.py

Loads the spaCy model lazily on first call. Processes the transcript text and returns an ordered dictionary of lemmas to frequency counts. Preserves first-appearance order. Filters by part of speech but not by length or stop-word status.

deck.py

Communicates with AnkiConnect via HTTP. Fetches all card fronts from the user's selected deck. Applies the three-condition fuzzy match to classify each lemma as skip, queue, or new. Writes queued words to the SQLite backlog when Anki is unavailable. Handles the interactive CLI prompt for queued words.

definition.py

Fetches definitions using a dual-source strategy. Always queries dictionaryapi.dev in the native transcript language for example sentences, synonyms, and antonyms. Queries Merriam-Webster or dictionaryapi.dev in the target output language for the definition and part of speech. Supplements missing fields across sources. Caches all results to SQLite.

translation.py

Translates a lemma from the transcript language to the definition output language. Tries community LibreTranslate mirrors first, then a locally installed argostranslate model. Prompts the user once per run if neither is available. Enforces a per-word timeout to prevent CPU hangs on local inference.

cards.py

Builds a genanki Model with ten named fields. Constructs genanki Notes from DefinitionResult objects. Generates stable GUIDs from the lemma and video ID. Writes the Deck and all Notes to a .apkg file with a timestamped filename.

state.py

Owens all pipeline-level SQLite tables: processed_videos, generated_packages, and vocabulary. Provides the in-memory Session object for deck selection. Does not own the definitions or anki_backlog tables which remain in their respective modules for fault isolation.

__main__.py

Parses CLI arguments, runs language resolution, dispatches to the appropriate pipeline mode (default, review, or backlog), orchestrates the module call sequence, presents the summary output, and offers the Anki import prompt.

3. Data Flow

The pipeline processes one video in a linear sequence. The CLI resolves the language code and deck name, then calls transcript.get_transcript() which returns a TranscriptResult. The full text is passed to nlp.process_transcript() which returns a vocabulary dictionary. The vocabulary is passed to deck.check_vocabulary() which returns a DeckCheckResult with skip, queue, and new lists. The new list plus any user-approved queue items are passed to definition.fetch_definitions() which returns a DefinitionBatchResult. The found list is passed to cards.build_package() which returns a PackageResult containing the path and accurate card counts. Finally state.mark_video_processed() records the run.

4. Storage

4.1 SQLite Database

The pipeline uses a single SQLite database file at the path specified by `DB_PATH` in the environment. The database contains five tables across three modules. The definitions table in `definition.py` uses a composite `lemma::language` key to store results by language pair. The `anki_backlog` table in `deck.py` stores words that could not be checked because Anki was not running. The `processed_videos`, `generated_packages`, and `vocabulary` tables in `state.py` track pipeline runs and vocabulary data for Phase 3.

4.2 File Output

Each pipeline run produces one `.apkg` file in the `OUTPUT_DIR` directory. The filename is the video ID plus a timestamp. Multiple runs of the same video produce separate files. The `.apkg` format is a ZIP archive containing an SQLite collection database and any media files.

4.3 Review File

When the user defers queued words, they are written to `review.json` in the project directory. Each entry contains the lemma, the matched front, the confidence score, and a decision field. The user sets the decision to add or skip and runs `make review` to process their decisions.

5. External Dependencies

youtube-transcript-api

Transcript extraction from YouTube. No authentication required. Proxy support via Webshare or generic HTTP/HTTPS/SOCKS configuration.

spaCy (en_core_web_sm)

NLP processing. Provides tokenization, lemmatization, POS tagging, and stop-word detection. Model loaded once and cached for process lifetime.

Merriam-Webster Collegiate API

Primary English definition source. Requires a free API key. 1000 requests per day on the free tier. Results cached to SQLite to minimise usage.

dictionaryapi.dev

Multilingual definition source. No authentication required. Used for native-language example sentences, synonyms, and antonyms regardless of `DEF_LANG`. Used as English definition fallback when MW is exhausted.

AnkiConnect

Local HTTP API for the running Anki desktop application. Used for deck listing and card front retrieval. Requires Anki to be open during the deck check step.

genanki

Anki package generation. Produces .apkg files without requiring Anki to be running. Used for all card and package creation.

argostranlate / LibreTranslate

Optional translation for DEF_LANG mode. Community mirrors tried first. Local model installed via make translate-setup. Each language pair model is approximately 150MB.

rapidfuzz

Fuzzy string matching for duplicate detection. Provides WRatio and token_sort_ratio scorers used in the three-condition filter.

6. WSL Networking Note

When running the pipeline inside WSL2 with Anki running as a native Windows application, AnkiConnect is not reachable at localhost:8765 by default. This is because WSL2 runs in a separate virtual network namespace. The fix is to set webBindAddress to 0.0.0.0 in AnkiConnect's configuration and set ANKI_HOST in the .env file to the WSL gateway IP, found by running: `ip route | grep default`. This IP may change between WSL sessions. A permanent fix via WSL mirrored networking mode is available on Windows 11 23H2 and later.